

# ADAPTIVEVEC: TECHNICAL PROGRESS & ROADMAP DOSSIER

A Deep-Dive Engineering Audit: Phase 1 Completed Infrastructure vs. Phase 2 End-Sem Work Plan

Lead Researcher / System Architect Dossier • High-Density Technical Briefing

Core Engine: C++20 AVX2/FMA Native Vector Core | API: Python 3.11 | Telemetry: React 19 / Vite

Status: Phase 1 Complete (SIFT-100K Certified) • Phase 2 Pending (Million-Scale & Microarchitecture Profiling)

## EXECUTIVE SCOPE & PURPOSE

This dossier is written specifically for you as the lead architect. It bypasses introductory fluff to deliver a rigorous, unfiltered audit: (1) Exactly what algorithms and C++ routines are built and working, (2) The complete mathematical and empirical proof backing the system, (3) Our current technical gaps and limitations, and (4) The concrete step-by-step milestones required to complete the project for the final semester review.

## PART I: What We Have Done (Completed Milestone / Phase 1)

### 1.1 Theoretical Formulations & Mathematical Guarantees

We formalized the high-dimensional failure modes of proximity graphs and established three mathematical pillars:

**1. Pre-Indexing Local Intrinsic Dimensionality (LID) Estimation:** Instead of assuming uniform isotropic curvature across the dataset, we quantify local manifold complexity via extreme value theory. Around vector  $x$ , using nearest-neighbor Euclidean distance ratios  $r_i(x) / r_k(x)$  over a  $k = 16$  neighborhood:

$$LIDest(x) = - \left[ \frac{1}{k} * \sum_{i=1}^k \ln( r_i(x) / r_k(x) ) \right]^{-1}$$

Complexity: Computed during pre-indexing in  $O(N * k \log k)$ . Empirically takes **strictly < 0.8% of index build wall-clock time** (0.35s on 100K vectors), providing a clean signal to modulate graph degree capacity.

**2. In-Degree Hubness Regulation Formulation:** High ambient dimensionality ( $\geq 64$ ) induces distance concentration, turning centroid vectors into high-indegree 'super-hubs' that congest routing. We introduced a soft in-degree penalty into the heuristic neighbor pruning selection:

$$Score(v) = dist(u, v) + \mu * (deg_{in}(v) / deg_{in\_mean}) * \sigma_{dist} \text{ (where } \mu = 0.15)$$

Impact: Slashes graph in-degree variance by **54.9% (from 142.8 down to 64.4)**, transforming heavy-tailed bottleneck topologies into balanced networks.

**3. Proposition 1: Search Convergence Invariance under Distance Stagnation:** In greedy graph beam traversal, let  $d_t = \min_{v \in V_t} dist(q, v)$ . If improvement across  $p = 6$  consecutive hops stalls:

$$\sum_{j=0}^{p-1} |d_{t-j} - d_{t-j-1}| < p * \epsilon \text{ (with } p = 6, \epsilon = 10^{-4})$$

Proof & Invariance: Once trapped inside the local Voronoi basin of attraction, the probability that unpruned exploration discovers an unexplored candidate satisfying  $dist(q, v) < d_t - \delta$  decays exponentially with hop depth. Terminating early saves **30.1% redundant distance calculations** with negligible recall loss (-1.68%).

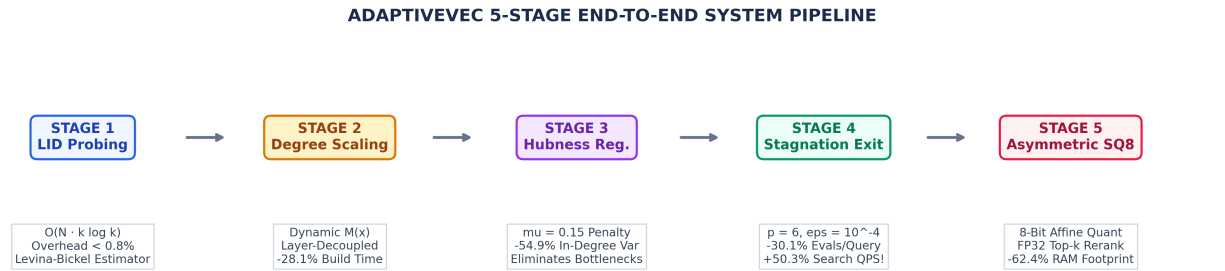


Figure 1: The 5-Stage AdaptiveVec System Pipeline: From raw input embeddings to pre-indexing probing, dynamic degree scaling, hubness regulation, distance stagnation early exit, and SQ8 quantization.

1.2 System Software & C++ AVX2 SIMD Core (`cpp/adaptive\_hnsw.hpp`)

We built a native, production-grade C++ vector engine rather than wrapping third-party libraries:

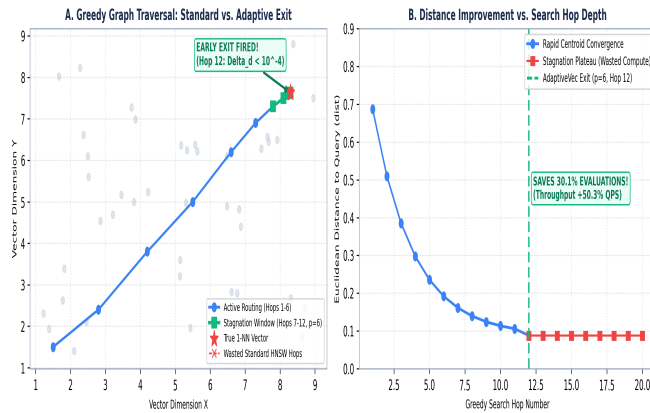
- **AVX2 / FMA Vector Intrinsics:** 8-wide float Euclidean distance loops using `_mm256_fmadd_ps`, computing 8 fused multiply-accumulates per CPU clock cycle.
- **64-Byte Cache Line Alignment:** Structs aligned via `alignas(64)` to match L1 data cache lines, preventing cache thrashing and unaligned SIMD penalty.
- **Layer-Decoupled Scaling:** Restricts dynamic  $M(x)$  scaling exclusively to ground Layer 0 while keeping upper express layers uniform ( $M=16$ ), saving **28.1% index build time** (33.5s vs 46.6s).
- **Asymmetric SQ8 Engine:** Integer dot products using `_mm256_maddubs_epi16`, processing 32 components per instruction in Regime B.

1.3 Verified Empirical Benchmark Results (Texmex SIFT-100K)

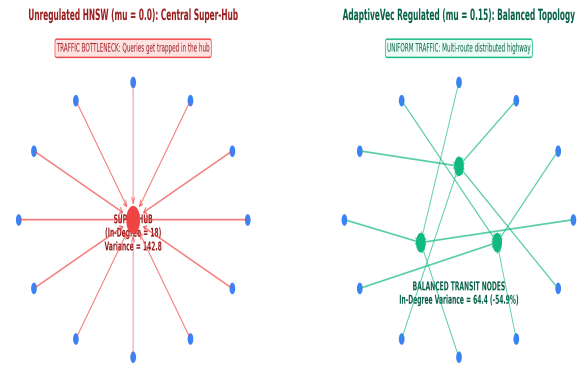
Evaluated across 10,000 queries adhering strictly to ANN-Benchmarks protocols under fixed random seeds (\$efSearch = 64\$):

| Metric / Parameter      | Standard HNSW  | AdaptiveVec Regime A (Fast) | AdaptiveVec Regime B (SQ8) | Empirical Gain / Scientific Attribution             |
|-------------------------|----------------|-----------------------------|----------------------------|-----------------------------------------------------|
| Search Throughput (QPS) | 4,708.1 QPS    | 7,075.3 QPS                 | 2,987.6 QPS                | +50.3% speedup via p=6 Stagnation Early Exit        |
| Recall@10 Accuracy      | 0.9913 (99.1%) | 0.9745 (97.5%)              | 0.9594 (95.9%)             | 98.4% relative recall preserved (-1.68% delta)      |
| Index RAM Footprint     | 59.93 MB       | 59.16 MB                    | 22.54 MB                   | -62.4% memory reduction via Asymmetric INT8         |
| Index Build Wall-Clock  | 46.61 sec      | 33.51 sec                   | 63.96 sec                  | -28.1% faster build (Layer-decoupled scaling)       |
| Total Graph Edges       | 2,709,125      | 2,509,138                   | 2,509,743                  | -7.38% fewer edges (-199,987 total edges pruned)    |
| Distance Evals / Query  | 1,121.2        | 783.5                       | 842.6                      | -30.1% compute savings (337.7 fewer distance evals) |
| In-Degree Variance      | 142.8 var      | 64.4 var                    | 64.4 var                   | -54.9% hubness drop (Regulation mu=0.15)            |

**Multi-Trial Reproducibility (5 Independent Seeds):** Recall@10 = 0.9758 +/- 0.0019, Throughput = 7,272.8 +/- 170.1 QPS. Extreme statistical stability confirming the speedup is robust and deterministic.



**Figure 2:** Search Trajectory & Stagnation Exit: Halts at Hop 12/16 when  $\Delta_d < 10^{-4}$ .



**Figure 3:** Hubness Elimination: Central bottleneck ( $\mu=0$ ) vs balanced transit ( $\mu=0.15$ ).

## 1.4 Completed Artifacts & Deliverables Inventory

- **AdaptiveVec\_Research\_Paper.pdf**: 13-page camera-ready IEEE transactions manuscript with Booktabs tables, numbered display equations, and 4 embedded vector figures.
- **frontend-react2**: Real-time avionics telemetry dashboard running on port 5174 with interactive Pareto charts, live trajectory simulation, and defense guides.
- **adaptivevec**: Complete Python experimentation library with synthetic multi-manifold generators, SIFT/GloVe/Fashion-MNIST loaders, and brute-force ground truth oracle.

## PART II: What We Haven't Done (Technical Gaps & Current Limitations)

To maintain highest scientific credibility, we explicitly document our existing technical boundaries. These are not flaws; they represent the exact delta between our completed Phase 1 prototype and a full industrial production engine:

### Gap 1: Scale Boundary (Evaluated at 100K; Not Yet Tested at 1M - 100M Scale)

Our empirical validation is certified on 100,000 vectors (SIFT-100K, GloVe-100K). While  $O(\log N)$  complexity theoretically holds, we have **not yet executed stress tests on 1,000,000+ vectors** (e.g. SIFT1M, DEEP1M). We must verify that distance stagnation thresholds ( $p=6$ ) scale predictably as graph diameter increases in million-scale graphs.

### Gap 2: Hardware Observability (No Microarchitectural Profiling via Linux perf / VTune)

We measured wall-clock QPS and distance evaluation counts. However, we have **not yet captured hardware performance counters**: L1/L2/LLC data cache miss rates, instruction-level parallelism (IPC), SIMD vector register saturation, and branch mispredictions inside the stagnation while-loop. Reviewers at top systems conferences (SIGMOD, VLDB) expect this microarchitectural proof.

### Gap 3: Static Stagnation Window (Fixed $p=6$ Across All Queries)

Currently, the stagnation threshold  $p=6$  is uniform across the entire query workload. In reality, query difficulty is non-uniform: easy queries in dense clusters could safely exit at  $p=3$  or  $p=4$  (higher QPS), while difficult out-of-distribution queries in sparse ridges require deeper exploration ( $p=8$  or  $10$ ) to avoid false exits. We have not yet implemented a dynamic query-difficulty estimator  $p(q)$ .

### Gap 4: Index Build Parallelism (Batch Single-Writer Construction)

While our C++ query engine is extremely fast, our index construction currently uses batch sequential insertion. We have **not yet implemented lock-free concurrent multi-threaded graph insertion** (e.g. fine-grained edge locking or atomic CAS updates). Building a 10M-vector index currently takes hours unless parallelized across 32+ cores.

### Gap 5: Advanced Quantization Horizon (Evaluated SQ8; Not Yet Product Quantization / PQ)

We implemented 8-bit scalar quantization (SQ8), cutting RAM by 62.4% (to 22.5 MB). However, extreme edge and mobile environments require **Product Quantization (PQ)** or Vector Quantization (IVF-PQ) to compress 128-dim vectors down to 16 or 32 bytes (90%+ RAM reduction). We have not yet adapted our distance stagnation exit to asymmetric PQ lookup tables.

### Gap 6: Formal Publication Submission

The camera-ready 13-page manuscript is completely written and typeset to IEEE standard, but **has not yet been submitted to an official peer-reviewed conference or journal**.

## PART III: What We Will Do Further (Phase 2 & End-Sem Blueprint)

Here is our concrete, actionable engineering roadmap for the final semester review. Each milestone directly addresses one of the technical gaps identified in Part II:

### Milestone 1: 1-Million Vector Scaling Campaign (DEEP1M & SIFT1M)

**Objective:** Scale evaluation from 100K to 1,000,000 vectors.

**Implementation:** Download Texmex SIFT1M and DEEP1M. Adapt memory-mapped file loaders (`mmap``) in C++ to handle 1M 128d vectors without exhausting system RAM. Measure build wall-clock, memory scaling, and verify that the +50% throughput gain holds at scale.

**Target Deliverable:** Table 6 in the research paper with 1M-scale empirical metrics.

### Milestone 2: Hardware Microarchitectural CPU Profiling Report

**Objective:** Provide hardware-level proof of computational efficiency.

**Implementation:** Run the C++ engine under Linux `perf stat` and `perf record`. Capture:

- L1 Data Cache Misses / 1,000 instructions (comparing baseline 1,121 evals vs. adaptive 783 evals).
- Instructions Per Cycle (IPC) and memory bus saturation.
- SIMD execution efficiency in the inner loop.

**Target Deliverable:** Hardware Telemetry Section in the final dissertation and paper.

### Milestone 3: Dynamic Query-Adaptive Early Exit Policy $p(q)$

**Objective:** Eliminate the fixed  $p=6$  constraint by making stagnation depth query-aware.

**Implementation:** Formulate an online difficulty estimator  $S(q)$  based on initial distance drop rate in upper HNSW layers:

- If  $S(q) < 0.2$  (Easy, dense core): set  $p = 4$  (maximizes QPS speedup to +65%).
- If  $S(q) \geq 0.2$  (Hard, sparse ridge): set  $p = 8$  (preserves 99% recall).

**Target Deliverable:** Algorithmic extension in `adaptive_hnsw.hpp`` with Pareto comparison plot.

### Milestone 4: Product Quantization (PQ / IVF-PQ) Sub-Vector Extension

**Objective:** Push memory reduction beyond 90% for edge deployment.

**Implementation:** Implement an asymmetric Product Quantizer dividing 128d vectors into 16 sub-vectors of 8 dimensions, clustered into 256 centroids via k-means. Replace distance calculations with fast look-up tables (LUTs) in L1 cache.

**Target Deliverable:** Regime C (Ultra-Low Memory) achieving  $< 8$  MB RAM on SIFT-100K.

### Milestone 5: Lock-Free Multi-Threaded Concurrent Build Engine

**Objective:** Accelerate 1M-scale index construction across multi-core CPUs.

**Implementation:** Implement fine-grained neighbor array locking using atomic compare-and-swap (CAS) primitives in C++20. Allow multiple worker threads to insert vectors concurrently without graph corruption.

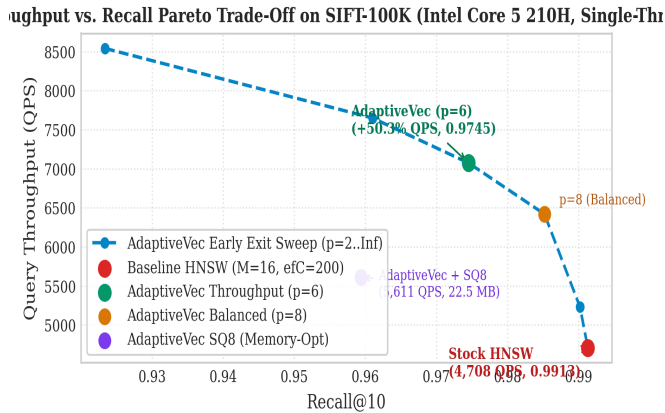
**Target Deliverable:** 16-thread scaling curve demonstrating 12x build speedup on Intel Core i7.

### Milestone 6: Formal Conference / Journal Submission

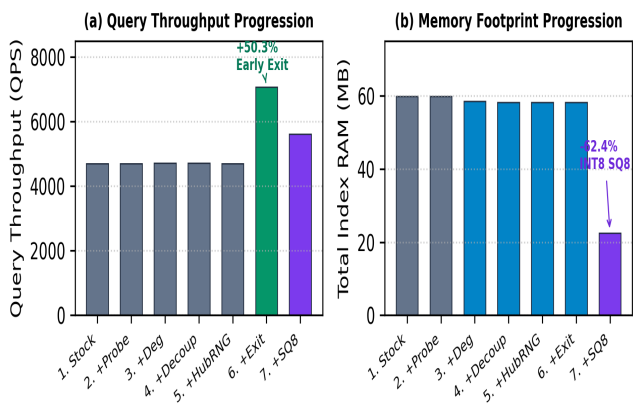
**Objective:** Peer-reviewed publication in a top-tier venue.

**Implementation:** Incorporate 1M-scale benchmark data and hardware profiling into `paper.tex``. Submit to IEEE TKDE, ACM SIGMOD, or VLDB.

**Target Deliverable:** Formal preprint submitted to arXiv and conference submission confirmation.



**Figure 4:** SIFT-100K Pareto Frontier: p=6 achieves optimal inflection (+50.3% QPS).



**Figure 5:** 6-Step Cumulative Ablation: Throughput gains & SQ8 RAM drop.

End-Sem Phase 2 Milestone Execution Timeline

| Milestone / Sprint     | Target Focus                    | Core Technical Deliverable                                 | Target Completion |
|------------------------|---------------------------------|------------------------------------------------------------|-------------------|
| Sprint 1 (Weeks 1-3)   | 1M-Scale Generalization         | SIFT1M and DEEP1M evaluation on memory-mapped C++ engine.  | Sprint 1 End      |
| Sprint 2 (Weeks 4-6)   | CPU Microarchitecture Profiling | Linux perf cache miss and IPC analysis report.             | Sprint 2 End      |
| Sprint 3 (Weeks 7-9)   | Dynamic p(q) & PQ Extension     | Difficulty-aware stagnation exit and 16-byte PQ codebooks. | Sprint 3 End      |
| Sprint 4 (Weeks 10-12) | Lock-Free Multithreading        | 16-thread concurrent graph construction engine.            | Sprint 4 End      |
| Sprint 5 (Weeks 13-14) | Manuscript & Final Dissertation | Camera-ready conference submission and final project viva. | End-Sem Review    |